Oracle PL/Sql

widoki, funkcje, procedury, triggery

ćwiczenie 1

Imiona i nazwiska autorów : Michał Kościanek, Michał Mąka

Tabele

- `Trip` - wycieczki
  - `trip_id` - identyfikator, klucz główny
  - `trip_name` - nazwa wycieczki
  - `country` - nazwa kraju
  - `trip_date` - data
  - `max_no_places` - maksymalna liczba miejsc na wycieczkę
- `Person` - osoby
  - `person_id` - identyfikator, klucz główny
  - `firstname` - imię
  - `lastname` - nazwisko
- `Reservation` - rezerwacje/bilety na wycieczkę
  - `reservation_id` - identyfikator, klucz główny
  - `trip_id` - identyfikator wycieczki
  - `person_id` - identyfikator osoby
  - `status` - status rezerwacji
    - `N` – New - Nowa
    - `P` – Confirmed and Paid – Potwierdzona i zapłacona
    - `C` – Canceled - Anulowana
- `Log` - dziennik zmian statusów rezerwacji
  - `log_id` - identyfikator, klucz główny
  - `reservation_id` - identyfikator rezerwacji
  - `log_date` - data zmiany
  - `status` - status

```
create sequence s_person_seq
   start with 1
   increment by 1;
```

```sql
create table person
(
  person_id int not null
      constraint pk_person
         primary key,
  firstname varchar(50),
  lastname varchar(50)
)

alter table person
    modify person_id int default s_person_seq.nextval;
```

```sql
create sequence s_trip_seq
   start with 1
   increment by 1;

create table trip
(
  trip_id int  not null
     constraint pk_trip
         primary key,
  trip_name varchar(100),
  country varchar(50),
  trip_date date,
  max_no_places int
);

alter table trip
    modify trip_id int default s_trip_seq.nextval;
```

```sql
create sequence s_reservation_seq
   start with 1
   increment by 1;

create table reservation
(
  reservation_id int not null
      constraint pk_reservation
         primary key,
  trip_id int,
  person_id int,
  status char(1)
);

alter table reservation
    modify reservation_id int default s_reservation_seq.nextval;
```

```
alter table reservation
add constraint reservation_fk1 foreign key
( person_id ) references person ( person_id );

alter table reservation
add constraint reservation_fk2 foreign key
( trip_id ) references trip ( trip_id );

alter table reservation
add constraint reservation_chk1 check
(status in ('N','P','C'));
```

```
create sequence s_log_seq
   start with 1
   increment by 1;


create table log
(
    log_id int not null
         constraint pk_log
         primary key,
    reservation_id int not null,
    log_date date not null,
    status char(1)
);

alter table log
    modify log_id int default s_log_seq.nextval;

alter table log
add constraint log_chk1 check
(status in ('N','P','C')) enable;

alter table log
add constraint log_fk1 foreign key
( reservation_id ) references reservation ( reservation_id );
```

Dane

Należy wypełnić tabele przykładowymi danymi

- 4 wycieczki
- 10 osób
- 10 rezerwacji

Dane testowe powinny być różnorodne (wycieczki w przyszłości, wycieczki w przeszłości, rezerwacje o różnym statusie itp.) tak, żeby umożliwić testowanie napisanych procedur.

W razie potrzeby należy zmodyfikować dane tak żeby przetestować różne przypadki.

```
-- trip
insert into trip(trip_name, country, trip_date, max_no_places)
values ('Wycieczka do Paryza', 'Francja', to_date('2023-09-12', 'YYYY-MM-DD'), 3);

insert into trip(trip_name, country, trip_date,  max_no_places)
values ('Piekny Krakow', 'Polska', to_date('2026-05-03','YYYY-MM-DD'), 2);

insert into trip(trip_name, country, trip_date,  max_no_places)
values ('Znow do Francji', 'Francja', to_date('2026-05-01','YYYY-MM-DD'), 2);

insert into trip(trip_name, country, trip_date,  max_no_places)
values ('Hel', 'Polska', to_date('2026-05-01','YYYY-MM-DD'),  2);

-- person
insert into person(firstname, lastname)
values ('Jan', 'Nowak');

insert into person(firstname, lastname)
values ('Jan', 'Kowalski');

insert into person(firstname, lastname)
values ('Jan', 'Nowakowski');

insert into person(firstname, lastname)
values  ('Novak', 'Nowak');

-- reservation
-- trip1
insert  into reservation(trip_id, person_id, status)
values (1, 1, 'P');

insert into reservation(trip_id, person_id, status)
values (1, 2, 'N');

-- trip 2
insert into reservation(trip_id, person_id, status)
values (2, 1, 'P');

insert into reservation(trip_id, person_id, status)
values (2, 4, 'C');

-- trip 3
insert into reservation(trip_id, person_id, status)
values (2, 4, 'P');
```

proszę pamiętać o zatwierdzeniu transakcji

Zadanie 0 - modyfikacja danych, transakcje,

wykonał: Michał Kościanek | review: Michał Mąka

Należy przeprowadzić kilka eksperymentów związanych ze wstawianiem, modyfikacją i usuwaniem danych oraz wykorzystaniem transakcji

Skomentuj dzialanie transakcji. Jak działa polecenie `commit`, `rollback`?. Co się dzieje w przypadku wystąpienia błędów podczas wykonywania transakcji? Porównaj sposób programowania operacji wykorzystujących transakcje w Oracle PL/SQL ze znanym ci systemem/językiem MS Sqlserver T-SQL

pomocne mogą być materiały dostępne są w UPEL:

https://upel.agh.edu.pl/mod/folder/view.php?id=411834

w szczególności dokumenty: `10_modyf_ora_north.pdf`, `20_ora_plsql_north.pdf`

```
BEGIN
-- insertujemy do tabeli reservation poprawne wartości istniejących:wycieczki,osoby,statusu
    INSERT INTO reservation(trip_id, person_id, status)
    values (1,5,'P');
-- tutaj commit poprawnie działa, zmiany są potwierdzane i wysyłane do bazy danych
COMMIT;
EXCEPTION
    WHEN OTHERS THEN
ROLLBACK;
end;

BEGIN
-- tutaj chcemy dodać do tabeli  reservation  raz nieistniejący trip_id
-- a potem trip_id:=null oraz osobe o nieistniejącym person_id
    INSERT INTO reservation(trip_id, person_id, status)
    values (50,2,'P');
    INSERT INTO reservation(trip_id, person_id, status)
    values (null,20,'P');
COMMIT;
-- wyjątek się wykonuje
EXCEPTION
    WHEN OTHERS THEN
-- zmiany są wycofywane
        ROLLBACK;
        RAISE;
end;
```

Zadanie 1 - widoki

Tworzenie widoków. Należy przygotować kilka widoków ułatwiających dostęp do danych. Należy zwrócić uwagę na strukturę kodu (należy unikać powielania kodu)

Widoki:

- `vw_reservation`
  - widok łączy dane z tabel: `trip`, `person`, `reservation`
  - zwracane dane: `reservation_id`, `country`, `trip_date`, `trip_name`, `firstname`, `lastname`, `status`, `trip_id`, `person_id`
- `vw_trip`
  - widok pokazuje liczbę wolnych miejsc na każdą wycieczkę
  - zwracane dane: `trip_id`, `country`, `trip_date`, `trip_name`, `max_no_places`, `no_available_places` (liczba wolnych miejsc)
- `vw_available_trip`
  - podobnie jak w poprzednim punkcie, z tym że widok pokazuje jedynie dostępne wycieczki (takie które są w przyszłości i są na nie wolne miejsca)

Proponowany zestaw widoków można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze widoki, funkcje
- np. można zmienić def. widoków, dodając nowe/potrzebne pola

Zadanie 1 - rozwiązanie

wykonał: Michał Mąka | review: Michał Kościanek

```sql
-- vw_reservation
CREATE OR REPLACE VIEW vw_reservation AS
SELECT
    reservation_id,
    country,
    trip_date,
    trip_name,
    firstname,
    lastname,
    status,
    t.trip_id,
    p.person_id
FROM reservation r
JOIN trip t ON r.trip_id = t.trip_id
JOIN person p ON r.person_id = p.person_id;

-- vw_trip - pokazanie wolnych miejsc (C - niezliczone)

CREATE OR REPLACE VIEW vw_trip AS
SELECT
    trip_id,
    country,
    trip_date,
    trip_name,
    max_no_places,
    (max_no_places - (
```

```sql
        SELECT COUNT(*)
        FROM reservation r
        WHERE r.trip_id = t.trip_id and r.status in ('N', 'P')
    )) AS no_available_places
FROM trip t;

-- vw_avaliable_trip - vw_trip with time constraint

CREATE OR REPLACE VIEW vw_available_trip AS
SELECT
    trip_id,
    country,
    trip_date,
    trip_name,
    max_no_places,
    no_available_places
FROM vw_trip
WHERE trip_date > SYSDATE
AND no_available_places > 0;
```

Zadanie 2 - funkcje

Tworzenie funkcji pobierających dane/tabele. Podobnie jak w poprzednim przykładzie należy przygotować kilka funkcji ułatwiających dostęp do danych

Procedury:

- `f_trip_participants`
  - zadaniem funkcji jest zwrócenie listy uczestników wskazanej wycieczki
  - parametry funkcji: `trip_id`
  - funkcja zwraca podobny zestaw danych jak widok `vw_reservation`
- `f_person_reservations`
  - zadaniem funkcji jest zwrócenie listy rezerwacji danej osoby
  - parametry funkcji: `person_id`
  - funkcja zwraca podobny zestaw danych jak widok `vw_reservation`
- `f_available_trips_to`
  - zadaniem funkcji jest zwrócenie listy wycieczek do wskazanego kraju, dostępnych w zadanym okresie czasu (od `date_from` do `date_to`)
    - dostępnych czyli takich na które są wolne miejsca
  - parametry funkcji: `country`, `date_from`, `date_to`

Funkcje powinny zwracać tabelę/zbiór wynikowy. Należy rozważyć dodanie kontroli parametrów, (np. jeśli parametrem jest `trip_id` to można sprawdzić czy taka wycieczka istnieje). Podobnie jak w przypadku widoków należy zwrócić uwagę na strukturę kodu

Czy kontrola parametrów w przypadku funkcji ma sens?

- jakie są zalety/wady takiego rozwiązania?

Proponowany zestaw funkcji można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze funkcje/procedury

Zadanie 2 - rozwiązanie

wykonał: Michał Kościanek | review: Michał Mąka

```oracle_plsql
-- Tworze typ obiektowy reprezentujący pojedynczy wiersz rezerwacji
CREATE OR REPLACE TYPE t_reservation_rec AS OBJECT (
    reservation_id INT, country VARCHAR2(50), trip_date DATE, trip_name VARCHAR2(100),
    firstname VARCHAR2(50), lastname VARCHAR2(50), status CHAR(1), trip_id INT, person_id INT
);


-- Tworzymy typ tabelaryczny bazujący na typie wierszu powyżej
CREATE OR REPLACE TYPE t_reservation_tab IS TABLE OF t_reservation_rec;


-- Typy dla wycieczek
CREATE OR REPLACE TYPE t_trip_rec AS OBJECT (
    trip_id INT, country VARCHAR2(50), trip_date DATE, trip_name VARCHAR2(100),
    max_no_places INT, no_available_places INT
);
-- ten sam zabieg co wczesniej
CREATE OR REPLACE TYPE t_trip_tab IS TABLE OF t_trip_rec;


CREATE OR REPLACE FUNCTION f_trip_participants(p_trip_id INT)
RETURN t_reservation_tab PIPELINED
IS
    v_count INT;
BEGIN
    -- sprawdzenie czy mamy pasujące wyniki
    SELECT COUNT(*) INTO v_count FROM trip WHERE trip_id = p_trip_id;
    IF v_count = 0 THEN
        RAISE_APPLICATION_ERROR(-20001, 'Wycieczka o podanym ID nie istnieje.');
    END IF;
    -- zwrócenie poprawnego wyniku
    FOR curr IN (SELECT * FROM vw_reservation WHERE trip_id = p_trip_id) LOOP
        PIPE ROW(t_reservation_rec(curr.reservation_id, curr.country, curr.trip_date,
                 curr.trip_name, curr.firstname, curr.lastname,
                 curr.status, curr.trip_id, curr.person_id));
    END LOOP;
    RETURN;
END;


CREATE OR REPLACE FUNCTION f_person_reservations(p_person_id INT)
RETURN t_reservation_tab PIPELINED
```

```
IS
BEGIN
    -- Pominąłem kontrolę parametru dla zwięzłości, ale można ją dodać analogicznie
    FOR curr IN (SELECT * FROM vw_reservation WHERE person_id = p_person_id) LOOP
        PIPE ROW(t_reservation_rec(curr.reservation_id, curr.country, curr.trip_date,
                 curr.trip_name, curr.firstname, curr.lastname,
                 curr.status, curr.trip_id, curr.person_id));
    END LOOP;
    RETURN;
END;

CREATE OR REPLACE FUNCTION f_available_trips_to(
    p_country VARCHAR2,
    p_date_from DATE,
    p_date_to DATE
    )
RETURN t_trip_tab PIPELINED
IS
BEGIN
-- sprawdzenie czy data rozpoczecia wycieczki nie jest po dacie zakonczenia
    IF p_date_from > p_date_to THEN
        RAISE_APPLICATION_ERROR(-20002, 'Data OD nie może być późniejsza niż data DO.');
    END IF;
-- zwrocenie tabeli poprawnych wyników
    FOR curr IN (
        SELECT * FROM vw_available_trip
        WHERE country = p_country
          AND trip_date BETWEEN p_date_from AND p_date_to
    ) LOOP
        PIPE ROW(t_trip_rec(curr.trip_id, curr.country, curr.trip_date,
                 curr.trip_name, curr.max_no_places, curr.no_available_places));
    END LOOP;
    RETURN;
END;
```

Zadanie 3 - procedury

Tworzenie procedur modyfikujących dane. Należy przygotować zestaw procedur pozwalających na modyfikację danych oraz kontrolę poprawności ich wprowadzania

Procedury

- `p_add_reservation`
    - zadaniem procedury jest dopisanie nowej rezerwacji
    - parametry: `trip_id`, `person_id`
    - procedura powinna kontrolować czy wycieczka jeszcze się nie odbyła, i czy sa wolne miejsca
    - procedura powinna również dopisywać inf. do tabeli `log`

- `p_modify_reservation_status`
  - zadaniem procedury jest zmiana statusu rezerwacji
  - parametry: `reservation_id`, `status`
  - dopuszczalne są wszystkie zmiany statusu
    - ale procedura powinna kontrolować czy taka zmiana jest możliwa, np. zmiana statusu już anulowanej wycieczki (przywrócenie do stanu aktywnego nie zawsze jest możliwa – może już nie być miejsc)
  - procedura powinna również dopisywać inf. do tabeli `log`
- `p_modify_max_no_places`
  - zadaniem procedury jest zmiana maksymalnej liczby miejsc na daną wycieczkę
  - parametry: `trip_id`, `max_no_places`
  - nie wszystkie zmiany liczby miejsc są dozwolone, nie można zmniejszyć liczby miejsc na wartość poniżej liczby zarezerwowanych miejsc

Należy rozważyć użycie transakcji

- czy należy użyć `commit` wewnątrz procedury w celu zatwierdzenia transakcji
  - jakie są tego konsekwencje

Należy zwrócić uwagę na kontrolę parametrów (np. jeśli parametrem jest trip_id to należy sprawdzić czy taka wycieczka istnieje, jeśli robimy rezerwację to należy sprawdzać czy są wolne miejsca itp..)

Proponowany zestaw procedur można rozbudować wedle uznania/potrzeb

- np. można dodać nowe/pomocnicze funkcje/procedury

Zadanie 3 - rozwiązanie

wykonał: Michał Mąka | review: Michał Kościanek

```
CREATE OR REPLACE PROCEDURE p_add_reservation(
    input_trip_id INT,
    input_person_id INT
)
IS
    var_available_places INT;
    var_trip_date DATE;
    var_log_reservation_id INT;
    var_person_check VARCHAR2(100);
BEGIN
    BEGIN
        SELECT no_available_places, trip_date
            INTO var_available_places, var_trip_date
        FROM vw_trip
        WHERE trip_id = input_trip_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Wycieczka nie istnieje.');
```

```
    END;

    IF var_available_places <= 0 THEN
        RAISE_APPLICATION_ERROR(-20011, 'Brak wolnych miejsc.');
    END IF;

    IF var_trip_date <= SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20012, 'Data wycieczki jest w przeszłości.');
    END IF;

    BEGIN
        SELECT LASTNAME INTO var_person_check
        FROM PERSON WHERE PERSON_ID = input_person_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20013, 'Osoba nie istnieje.');
    END;

    INSERT INTO RESERVATION (trip_id, person_id, status)
    VALUES (input_trip_id,input_person_id,'N')
    RETURNING RESERVATION_ID INTO var_log_reservation_id;

    INSERT INTO LOG (RESERVATION_ID, LOG_DATE, STATUS)
    VALUES (var_log_reservation_id, SYSDATE, 'N');
END;


-- p_modify_reservation_status
CREATE OR REPLACE PROCEDURE p_modify_reservation_status (
    input_reservation_id INT,
    input_status CHAR
    )
IS
    var_old_status CHAR;
    var_trip_id INT;
    var_available_places INT;
BEGIN
    BEGIN
        SELECT status, trip_id INTO var_old_status, var_trip_id
        FROM reservation WHERE reservation_id = input_reservation_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20021, 'Nie istnieje rezerwacja o podanym ID.');
    END;

    -- przywracanie z anulowanej
    IF var_old_status = 'C' AND input_status IN ('N', 'P') THEN
        SELECT no_available_places INTO var_available_places
        FROM vw_trip WHERE trip_id = var_trip_id;

        IF var_available_places <= 0 THEN
            RAISE_APPLICATION_ERROR(-20020, 'Brak wolnych miejsc.');
        END IF;
    END IF;
```

```
    UPDATE reservation
    SET status = input_status
    WHERE reservation_id = input_reservation_id;

    -- sql%rowcount - ile zmienilo sie wierszy przez procedure
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20021, 'Nie istnieje rezerwacja o podanym ID.');
    END IF;

    INSERT INTO log (reservation_id, log_date, status)
    VALUES (input_reservation_id, SYSDATE, input_status);
END;


-- p_modify_max_no_places
CREATE OR REPLACE PROCEDURE p_modify_max_no_places(
    input_trip_id INT,
    input_no_places INT)
IS
    var_taken_places INT;
    var_trip_exists INT; -- do sprawdzenia istnienia danej wycieczki
BEGIN
    SELECT COUNT(*) INTO var_taken_places
    FROM reservation
    WHERE trip_id = input_trip_id AND status IN ('N', 'P');

    IF input_no_places < var_taken_places THEN
        RAISE_APPLICATION_ERROR(-20030,
            'Próba zmiany ilości miejsc na mniejszą niż liczba zarezerwowanych.');
    END IF;


    SELECT COUNT(*) INTO var_trip_exists FROM trip WHERE trip_id = input_trip_id;

    IF var_trip_exists = 0 THEN
        RAISE_APPLICATION_ERROR(-20031, 'Nie ma wycieczki o danym ID.');
    END IF;

    UPDATE trip SET max_no_places = input_no_places WHERE trip_id = input_trip_id;

END;
```

Zadanie 4 - triggery

Zmiana strategii zapisywania do dziennika rezerwacji. Realizacja przy pomocy triggerów

Należy wprowadzić zmianę, która spowoduje, że zapis do dziennika będzie realizowany przy pomocy trigerów

Triggery:

- trigger/triggery obsługujące
  - dodanie rezerwacji
  - zmianę statusu
- trigger zabraniający usunięcia rezerwacji

Oczywiście po wprowadzeniu tej zmiany należy "uaktualnić" procedury modyfikujące dane.

> UWAGA Należy stworzyć nowe wersje tych procedur (dodając do nazwy dopisek 4 - od numeru zadania). Poprzednie wersje procedur należy pozostawić w celu umożliwienia weryfikacji ich poprawności

Należy przygotować procedury: `p_add_reservation_4`, `p_modify_reservation_status_4` , `p_modify_reservation_4`

Zadanie 4 - rozwiązanie

wykonał: Michał Kościanek | review: Michał Mąka

```oracle_plsql
-- Trigger akutalizując zmiany do tabeli log
CREATE OR REPLACE TRIGGER trg_reservation_log
AFTER INSERT OR UPDATE OF status ON reservation
FOR EACH ROW
BEGIN
    INSERT INTO log (reservation_id, log_date, status)
    VALUES (:NEW.reservation_id, SYSDATE, :NEW.status);
END;


-- Trigger zabraniający usuwania rezerwacji
CREATE OR REPLACE TRIGGER trg_reservation_prevent_del
BEFORE DELETE ON reservation
FOR EACH ROW
BEGIN
    RAISE_APPLICATION_ERROR(-20040,
        'Usuwanie rezerwacji jest zablokowane. Zmień status na C (Canceled).');
END;


CREATE OR REPLACE PROCEDURE p_add_reservation_4(
    input_trip_id INT,
    input_person_id INT)
IS
    var_available_places INT;
    var_trip_date DATE;
BEGIN
    BEGIN
        SELECT no_available_places, trip_date
            INTO var_available_places, var_trip_date
        FROM vw_trip
```

```
        WHERE trip_id = input_trip_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Wycieczka nie istnieje.');
    END;

    -- wyjątek obycia się już wycieczki
    IF var_trip_date <= SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20011, 'Wycieczka już się odbyła lub trwa.');
    END IF;
    -- wyjatek przy brakujacych miejsach wycieczki
    IF var_available_places <= 0 THEN
        RAISE_APPLICATION_ERROR(-20012, 'Brak wolnych miejsc na wycieczkę.');
    END IF;

    -- Tabela log zaktualizuje się sama przez trigger
    INSERT INTO reservation (trip_id, person_id, status)
    VALUES (input_trip_id, input_person_id, 'N');
END;

CREATE OR REPLACE PROCEDURE p_modify_reservation_status_4(
    input_reservation_id INT,
    input_status CHAR)
IS
    var_old_status CHAR(1);
    var_trip_id INT;
    var_available_places INT;
BEGIN
    BEGIN
        SELECT status, trip_id INTO var_old_status, var_trip_id
        FROM reservation WHERE reservation_id = input_reservation_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20021, 'Nie istnieje rezerwacja o podanym ID.');
    END;

    IF var_old_status = 'C' AND input_status IN ('N', 'P') THEN
        SELECT no_available_places INTO var_available_places
        FROM vw_trip WHERE trip_id = var_trip_id;
        -- warunek sprawdzający czy można zmienić status z odwolanego na
        -- nowy/potwierdzony i oplacony spowodowany przez brak miejsc
        IF var_available_places <= 0 THEN
            RAISE_APPLICATION_ERROR(-20020,
                'Nie można przywrócić rezerwacji. Brak wolnych miejsc.');
        END IF;
    END IF;

    -- Log zostanie wpisany przez trigger
    UPDATE reservation SET status = input_status
    WHERE reservation_id = input_reservation_id;
END;
```

Zadanie 5 - triggery

Zmiana strategii kontroli dostępności miejsc. Realizacja przy pomocy triggerów

Należy wprowadzić zmianę, która spowoduje, że kontrola dostępności miejsc na wycieczki (przy dodawaniu nowej rezerwacji, zmianie statusu) będzie realizowana przy pomocy trigerów

Triggery:

- Trigger/triggery obsługujące:
  - dodanie rezerwacji
  - zmianę statusu

Oczywiście po wprowadzeniu tej zmiany należy "uaktualnić" procedury modyfikujące dane.

> UWAGA Należy stworzyć nowe wersje tych procedur (np. dodając do nazwy dopisek 5 - od numeru zadania). Poprzednie wersje procedur należy pozostawić w celu umożliwienia weryfikacji ich poprawności.

Należy przygotować procedury: `p_add_reservation_5`, `p_modify_reservation_status_5`, ...

Zadanie 5 - rozwiązanie

wykonał: Michał Mąka | review: Michał Kościanek

```oracle
-- trigger sprawdzający ilosc wolnych miejsc
CREATE OR REPLACE TRIGGER trg_check_available_places
FOR INSERT OR UPDATE OF status ON reservation
COMPOUND TRIGGER

    -- v_trips - lista ID wycieczek aktualizowanych, na wiele naraz
    TYPE t_trip_list IS TABLE OF INT INDEX BY PLS_INTEGER;
    v_trips t_trip_list;
    v_idx INT := 1;

    BEFORE EACH ROW IS
    BEGIN
        -- zliczamy jezeli dodane nowe ok rezerwacje
        IF INSERTING AND :NEW.status IN ('N', 'P') THEN
            v_trips(v_idx) := :NEW.trip_id;
            v_idx := v_idx + 1;
        -- zliczamy dodatkowo, jeżeli z stan zmienia sie z anulowanej
        ELSIF UPDATING AND :OLD.status = 'C' AND :NEW.status IN ('N', 'P') THEN
            v_trips(v_idx) := :NEW.trip_id;
            v_idx := v_idx + 1;
        END IF;
    END BEFORE EACH ROW;

    -- sprawdzenie po aktualizacji ile jest wolnych miejsc
```

```
    AFTER STATEMENT IS
        v_available_places INT;
    BEGIN
        FOR i IN 1 .. v_trips.COUNT LOOP
            SELECT no_available_places INTO v_available_places
            FROM vw_trip
            WHERE trip_id = v_trips(i);

            IF v_available_places < 0 THEN
                RAISE_APPLICATION_ERROR(-20050, 'Brak wolnych miejsc - trigger.');
            END IF;
        END LOOP;

    END AFTER STATEMENT;

END trg_check_available_places;


CREATE OR REPLACE PROCEDURE p_add_reservation_5(
    input_trip_id INT,
    input_person_id INT
)
IS
    var_trip_date DATE;
BEGIN
    BEGIN
        SELECT trip_date
            INTO var_trip_date
        FROM trip
        WHERE trip_id = input_trip_id;
    EXCEPTION
        WHEN NO_DATA_FOUND THEN
            RAISE_APPLICATION_ERROR(-20010, 'Wycieczka nie istnieje.');
    END;

    IF var_trip_date <= SYSDATE THEN
        RAISE_APPLICATION_ERROR(-20012, 'Data wycieczki jest w przeszłości.');
    END IF;

    INSERT INTO RESERVATION (trip_id, person_id, status)
    VALUES (input_trip_id,input_person_id,'N');
END;

CREATE OR REPLACE PROCEDURE p_modify_reservation_status_5(
    input_reservation_id INT,
    input_status CHAR
    )
IS
BEGIN
    UPDATE reservation
    SET status = input_status
    WHERE reservation_id = input_reservation_id;

    -- sql%rowcount - ile zmienilo sie wierszy przez procedure
```

```
    IF SQL%ROWCOUNT = 0 THEN
        RAISE_APPLICATION_ERROR(-20021, 'Nie istnieje rezerwacja o podanym ID.');
    END IF;
END;
```

Zadanie - podsumowanie

Porównaj sposób programowania w systemie Oracle PL/SQL ze znanym ci systemem/językiem MS Sqlserver T-SQL

```
Zmiany względem Ms Sqlserver T-SQL:

- Jawne commitowanie zmian do bazy (commit / rollback)
- brak `BEGIN TRANSACTION` przy transakcjach/procedurach
- w procedurach zmienne tylko na początku
- triggery po wierszach (FOR EACH ROW)
- obsluga :NEW, :OLD przed i po triggerze
```